EXP 10

```
clc;
clear;
close all;

% Performance Analysis of FM System under Gaussian White Noise

%% 1. System Parameters
Fs = 200000;          % Sampling frequency (Hz)
T = 0.1;              % Simulation time (seconds)
t = 0:1/Fs:T-1/Fs;

fm = 200;             % Message frequency (Hz)
fc = 10000;           % Carrier frequency (Hz)
Am = 1;               % Message amplitude
Ac = 1;               % Carrier amplitude
kf = 2000;            % Frequency sensitivity (Hz/V)

SNR_dB = [0 5 10 15 20]; % Input SNR values

%% 2. Generate Message Signal
m = Am*cos(2*pi*fm*t);

%% 3. Generate FM Signal
phase = 2*pi*fc*t + 2*pi*kf*cumtrapz(t,m);
sFM = Ac*cos(phase);

%% 4. Initialize Performance Variables
MSE = zeros(size(SNR_dB));
OutputSNR = zeros(size(SNR_dB));
```

```
% Low-pass filter for recovered message
[b,a] = butter(5, 1000/(Fs/2), 'low');

%% 5. Plot Original Message and FM Signal
figure('Name','FM Signal Generation');

subplot(2,1,1);
plot(t,m,'b');
title('Original Message Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(2,1,2);
plot(t,sFM,'r');
title('Frequency Modulated Signal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

%% 6. Noisy Channel and FM Demodulation
figure('Name','FM Performance Under AWGN');

for k = 1:length(SNR_dB)

    % Add Gaussian white noise
    signalPower = mean(sFM.^2);
    noisePower = signalPower / 10^(SNR_dB(k)/10);
    noise = sqrt(noisePower)*randn(size(sFM));
    received = sFM + noise;
```

```
% FM demodulation using phase differentiation
analyticSignal = hilbert(received);
unwrappedPhase = unwrap(angle(analyticSignal));

instFreq = [diff(unwrappedPhase)*Fs/(2*pi), 0];

% Remove carrier frequency
detected = instFreq - fc;

% Low-pass filtering
detected = filtfilt(b,a,detected);

% Remove numerical offset
detected = detected - mean(detected);

% Normalize to original message amplitude
detected = detected * ...
   (Am / (max(abs(detected)) + eps));

% Ignore edge samples during performance calculation
idx = round(0.01*Fs):round(0.09*Fs);
errorSignal = m(idx) - detected(idx);

% Calculate MSE and output SNR
MSE(k) = mean(errorSignal.^2);
OutputSNR(k) = 10*log10( ...
   mean(m(idx).^2)/(MSE(k) + eps));

% Plot a short section of detected signal
subplot(length(SNR_dB),1,k);
plot(t(idx),m(idx),'b');
```

```
    hold on;
    plot(t(idx),detected(idx),'r');
    title(sprintf('FM Detection at Input SNR = %d dB', ...
        SNR_dB(k)));
    xlabel('Time (s)');
    ylabel('Amplitude');
    legend('Original','Detected');
    grid on;
end

%% 7. Plot Performance Graphs
figure('Name','FM Performance Evaluation');

subplot(2,1,1);
semilogy(SNR_dB,MSE,'bo-','LineWidth',1.5);
title('MSE versus Input SNR');
xlabel('Input SNR (dB)');
ylabel('Mean Squared Error');
grid on;

subplot(2,1,2);
plot(SNR_dB,OutputSNR,'rs-','LineWidth',1.5);
title('Output SNR versus Input SNR');
xlabel('Input SNR (dB)');
ylabel('Output SNR (dB)');
grid on;

%% 8. Display Results
fprintf('\nFM Performance Analysis Results\n');
fprintf('----------------------------------------\n');
fprintf('Input SNR (dB)    MSE       Output SNR (dB)\n');
```

```
fprintf('----------------------------------------\n');

for k = 1:length(SNR_dB)
    fprintf('%10.1f    %8.5f    %10.2f\n', ...
        SNR_dB(k), MSE(k), OutputSNR(k));
end

disp('FM performance analysis completed successfully.');
```

